Low-Cost IoT-Based Multi-Sensor Air Quality Monitoring with Real-Time Cloud Visualization and Local Alerting

Diya Aggarwal[1], Arijit Majumder[2], Swarnalatha P[3]
[1,2,3]*School of Computer Science and Engineering, Vellore Institute of Technology, Vellore, Tamil Nadu, India – 632014*
[1]diya.aggarwal2023@vitstudent.ac.in
[2]arijit.majumder2023@vitstudent.ac.in
[3]pswarnalatha@vit.ac.in
Corresponding author: Swarnalatha P[3], [3]pswarnalatha@vit.ac.in

***Abstract*—**The issue of poor air quality is one of the least tangible problems in a rapidly growing city, and the monitoring station capable of exposing this problem is both expensive and rare. This paper discusses the creation and evaluation of a cost-effective air pollution monitoring node that has been built on Arduino Uno microcontroller board, DHT11 temperature-humidity sensor and MQ5 and MQ135 gas sensors. The data is sent through Wi-Fi connection to the ThingSpeak platform, where it is shown almost in real time. At the same time, buzzer, LED and 16×2 LCD notify people nearby about passing a certain threshold. The prototype was tested in real network conditions, and the results of tests, as well as some logs and spot checks, are given below. Temperature and AQI values were consistent with the reference data up to 1 °C and 3 index points accordingly, and the alert mechanism operated as expected.

***Index Terms*—**Air quality monitoring, Arduino, Internet of Things, smart city, ThingSpeak, MQ135, wireless sensor node.

I. INTRODUCTION

Cities pack people, vehicles and industry into a small space, and the air that results is often worse than what the same population would breathe elsewhere. Carbon monoxide (CO), methane (CH4), ammonia (NH3) and fine particles are all linked to asthma, bronchitis and cardiovascular disease [1], and the World Health Organization (WHO) estimates that almost nine out of ten children are exposed to particulate levels above its guideline [2]. It is therefore clear that knowing what is in the air at street and hour level is a necessity for any city to be regarded as a smart one.

The Internet of Things (IoT) makes that kind of awareness affordable. Small sensing boards can be networked, left unattended and read from anywhere, and the cloud takes care of storage and presentation [3], [4]. In the smart city literature, environmental sensing is regarded as one of the most natural IoT applications because the data is continuous, location-bound and useful to many parties at once [5].

Conventional monitoring, however, has not caught up. Reference-grade stations are expensive, heavy and installed at only a handful of points [6]. Many cheaper systems simply log data and leave the user to notice a problem later, and indoor air, where people spend most of their day, is rarely covered at all [7].

This work tries to close part of that gap. The contributions are as follows:

1) A low-cost sensor node for the measurement of temperature, humidity, and the reactions of CO, NH_3, CH_4, and ethanol-based gases.
2) An entire chain from sensors to the cloud-based dashboard through Wi-Fi in a 15-second fixed cycle time.
3) An on-site alerting module (with buzzer and LED), which operates regardless of any network availability.
4) An evaluation of the prototype against known reference values and an analysis of the application of the node within a city-wide infrastructure.

The rest of the paper is organized like this. The relevant literature review, research gaps and novelty of this study are presented in Section II, Problem Statement is in Section III, System Architecture in Section IV, Hardware Implementation in Section V, Software Part and Alerting Mechanism in Section VI, Results in Section VII, Smart City Application in Section VIII, and Conclusions in Section IX.

II. RELATED WORK

A number of groups have built IoT-based air monitors, and they differ mostly in the processor, the sensor set and how the data is delivered. Jayakumar [8] paired an Arduino with MQ-series gas sensors and showed that real-time monitoring is feasible at low cost, but the system had no way of anticipating trends. Gupta et al. [9] looked at the smart city case directly and proposed a network for city-wide coverage; their design, though, offered little in the way of immediate alerts. Singh et al. [10] built a gas detector on an Arduino, which kept the cost down but became awkward to extend to larger spaces.

Desai et al. [11] moved to a BeagleBone Black, gaining processing headroom at the price of a more expensive and more complex node. Phala et al. [12] followed a standards-based approach to air quality monitoring and obtained dependable measurements, although the work did not include a cloud dashboard. Pal et al. [13] built a continuous pollution logger without any analytical layer, and Dhingra et al. [14] described a mobile monitor whose hardware was only lightly validated.

Anitha and Kumar [15] combined a particulate sensor, gas sensors and filters on a Raspberry Pi 3 B+, so that the unit both measured and cleaned the air, and sent its readings to ThingSpeak. Kondeti et al. [4] reported a real-time air quality and weather station aimed at sustainable urban development.

Taken together, these studies show that sensing and logging are well covered. What is less consistent is the combination we are after: a cheap node that also raises a local alarm, publishes to a cloud dashboard, and can be multiplied across a city. That is the combination attempted here.

A. Research Gaps

Reading the studies above together with the way air quality is officially monitored today, through fixed reference stations such as those of the Central Pollution Control Board [16], five gaps stand out.

1) Scalability issues: reference stations and single node prototypes are both limited to one point and cannot be expanded at a reasonable cost to cover a city.
2) Calibration and accuracy: the MQ series sensors require preheating and must be frequently calibrated, and their values will drift if left uncalibrated for some time.
3) High cost and complexity: reference-grade stations, and boards such as the BeagleBone Black [11], raise cost and implementation effort, which limits wide deployment.
4) Weak integration of sensors with the cloud: many systems collect data locally but stop short of combining several sensors with real-time cloud visualization [12].
5) No low-cost real-time alerting: most setups log data for later review, and few give an immediate on-site warning when pollutant levels rise [9].

B. Novelty of the Proposed Work

The novelty of this work is that it addresses these gaps together in one node instead of one at a time. Four design choices carry it:

1) Fusion of multiple sensors: a combination of DHT11, MQ5 and MQ135 sensors on an Arduino board can measure the temperature and humidity as well as detect the presence of methane, CO, NH_3 and ethanol gases in one go rather than having a single gas detection.
2) Cloud-first approach: the use of an ESP8266 microcontroller allows us to send all the data readings to ThingSpeak, thereby allowing the system to be extended without being dependent on any one device.
3) Real-time local alerts: the buzzer and LED lights up as soon as any threshold value is reached.
4) Low-cost bill of materials: off-the-shelf components (Arduino Uno, MQ series sensors, ESP8266) allow the price to remain very low in comparison to reference stations.

Calibration drift (gap 2) is not eliminated by this design. It is managed through pre-heating, datasheet-based calibration and the validation in Section VII, and the remaining limits are listed in Section VII-E.

III. PROBLEM STATEMENT

There are various toxic chemicals that can be present in the environment that surrounds us, including CO, NH_3 and CH_4; however, the tools available to detect them are usually quite costly or bulky. Existing low-cost units often lack remote access or an immediate warning, so users learn about a hazard only after exposure. The problem addressed in this paper is the design of a portable, inexpensive node that (i) senses temperature, humidity and harmful gases continuously, (ii) makes the data available remotely in real time, and (iii) warns people on the spot when a pre-set limit is exceeded, while remaining simple enough to be replicated across many city locations.

Throughout the paper we express air quality using the Indian National Air Quality Index, which groups concentrations into six bands from “Good” to “Severe” [16]. Fig. 1 lists the categories and Fig. 2 the pollutant breakpoints on which the bands are based.

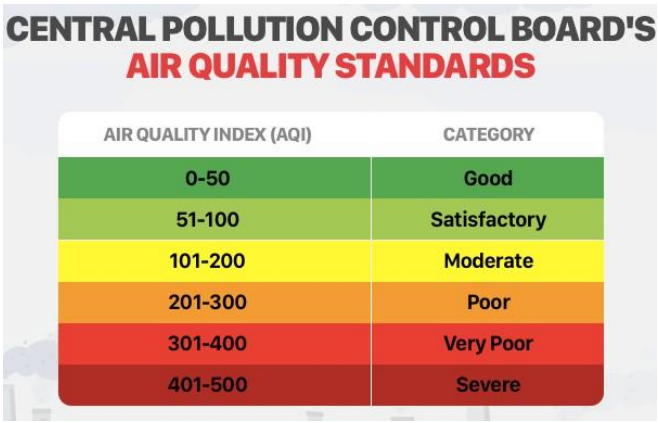


Fig. 1. CPCB air quality index categories used as the reference scale.

AQI Category (Range)	PM_{10} 24-hr	$PM_{2.5}$ 24-hr	NO_2 24-hr	O_3 8-hr	CO 8-hr (mg/m^3)	SO_2 24-hr	NH_3 24-hr	Pb 24-hr
Good (0-50)	0-50	0-30	0-40	0-50	0-1.0	0-40	0-200	0-0.5
Satisfactory (51-100)	51-100	31-60	41-80	51-100	1.1-2.0	41-80	201-400	0.6 –1.0
Moderate (101-200)	101-250	61-90	81-180	101-168	2.1- 10	81-380	401-800	1.1-2.0
Poor (201-300)	251-350	91-120	181-280	169-208	10.1-17	381-800	801-1200	2.1-3.0
Very poor (301-400)	351-430	121-250	281-400	209-748*	17.1-34	801-1600	1201-1800	3.1-3.5
Severe (401-500)	430 +	250+	400+	748+*	34+	1600+	1800+	3.5+

Fig. 2. AQI categories and the pollutant concentration ranges that correspond to them [16].

IV. SYSTEM ARCHITECTURE

A. Layered Design

The system is organized into five layers, shown in Fig. 3. shows how data flows from the sensors at the bottom up to the user-facing application at the top: sensors send data to the microcontroller, then to the communication module, then to the cloud, and finally to the dashboard. Keeping the layers separate means that any one of them, for example the Wi-Fi link, can be replaced later without redesigning the rest.

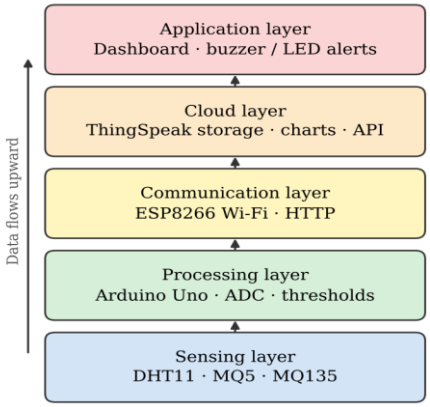


Fig. 3. Five-layer architecture of the proposed monitoring node.

B. Sensing Layer

Three sensors make up the sensing layer. A DHT11 reports temperature and relative humidity as a digital signal. An MQ5 responds mainly to methane and other combustible gases, while an MQ135 responds to ammonia, carbon monoxide, ethanol and other common pollutants. Both MQ devices are metal-oxide sensors: a heated sensing element changes its electrical resistance in proportion to the gas around it, and the board converts that change into an analog voltage [17], [18].

C. Processing Layer

The control unit used is an Arduino Uno, which has an ATmega328P microcontroller. It accepts both analog as well as digital signals, performs voltage calculations using the built-in 10-bit ADC, compares the received signal against the threshold value, and creates a data packet to transmit over the communication interface.

D. Communication Layer

Wi-Fi capabilities are provided by an ESP8266 module. The module periodically uploads data to the ThingSpeak API via HTTP requests; in the course of our experiments, the module successfully connected to a mobile Wi-Fi hotspot, proving thereby that the node is independent of any fixed network infrastructure.

E. Cloud and Application Layers

ThingSpeak [19] stores each reading, draws it as a time-series chart and exposes a REST interface for retrieving the data. The application layer is what users actually see: the web dashboard from any internet-connected device, and, at the node itself, an LCD together with a buzzer and LED that signal threshold violations. The hardware arrangement is given in Fig. 4 and the connections and pin assignments in Fig. 5.

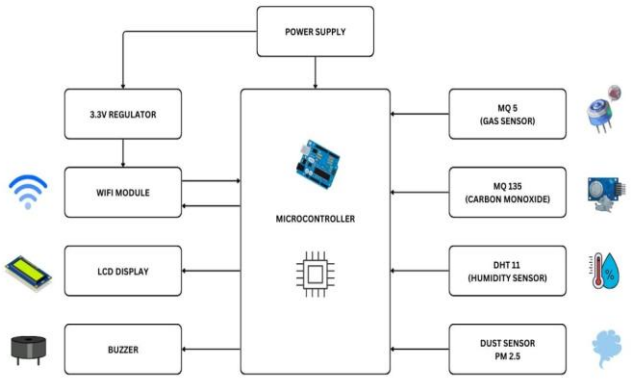


Fig. 4. Hardware system architecture of the monitoring node.

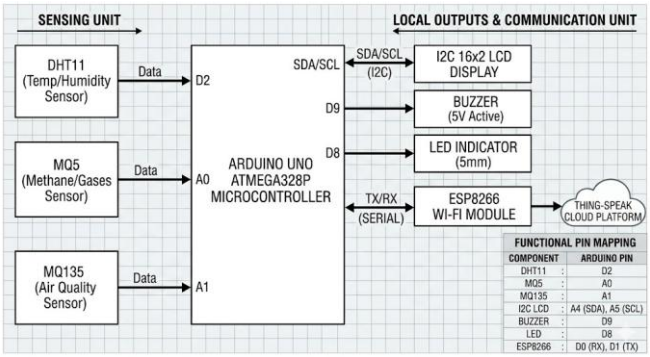


Fig. 5. Block diagram of the node with functional pin mapping on the Arduino Uno.

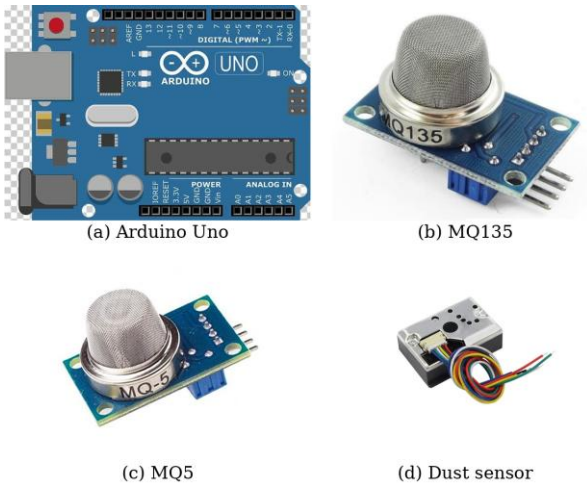
V. HARDWARE IMPLEMENTATION

A. Components

Table I lists the parts used. All are widely available hobby-grade modules, which is deliberate: the aim is a node that a municipality, a school or an individual could afford in quantity. Fig. 6 shows the main boards and sensors.

TABLE I
HARDWARE COMPONENTS

Component	Specification	Purpose
Arduino Uno	ATmega328P	Processing and control
MQ5 sensor	Analog output	Methane / combustible gases
MQ135 sensor	Analog output	CO, NH_3, ethanol, air quality
DHT11 sensor	Digital output	Temperature, humidity
ESP8266	Wi-Fi 802.11 b/g/n	Cloud data transfer
LCD	16×2 characters	Local display
Buzzer	5 V active	Audible alert
LED	5 mm	Visual alert



(a) Arduino Uno (b) MQ135

(c) MQ5 (d) Dust sensor

Fig. 6. Main modules of the prototype: (a) Arduino Uno, (b) MQ135 gas sensor, (c) MQ5 gas sensor and (d) optical dust sensor fitted for particulate measurement.

B. Circuit Configuration

The MQ5 and MQ135 outputs are wired to analog inputs A0 and A1, and the DHT11 data line to a digital pin (D2). The ESP8266 communicates with the Arduino over the serial lines, the buzzer and LED are driven from digital outputs D9 and D8, and the LCD is attached through the I²C bus (A4/A5). The complete schematic is in Fig. 7 and the assembled prototype in Fig. 8.

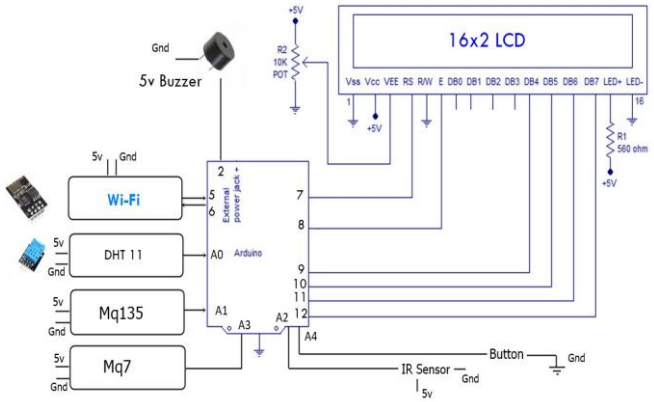


Fig. 7. Circuit diagram of the node.

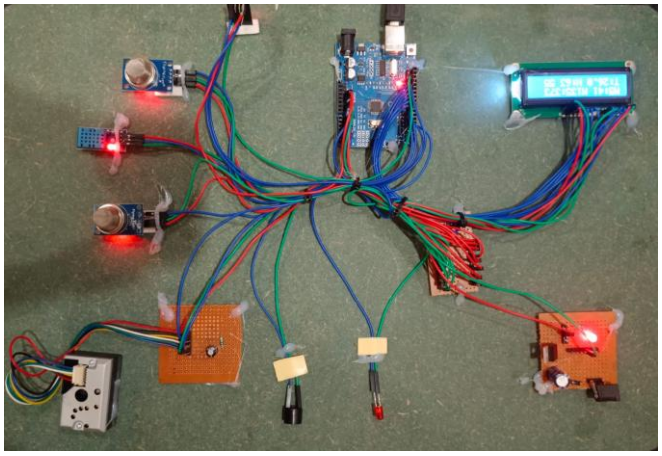


Fig. 8. Assembled prototype with gas sensors, DHT11, LCD, dust sensor and the alert indicators.

C. Calibration and Signal Conversion

The MQ sensors require a considerable period of time to heat up before providing steady output values, so all the devices were preheated for 24 hours before the first measurement, as recommended by the manufacturers. Calibration coefficients were taken from the datasheets and not measured using a gas chamber, while alarm thresholds were determined based on the air quality standards limits.

The 10-bit ADC maps the sensor voltage to an integer, and the voltage is recovered by

$$V = ADC \times 5 / 1023 \quad (1)$$

From this voltage the sensor resistance R_s is found using the load resistor R_L and the supply voltage V_c,

$$R_s = R_L\,(V_c - V) / V \qquad (2)$$

and the gas concentration follows from the log–log characteristic given in the datasheet, in which the ratio of R_s to the clean-air resistance R_0 is related to concentration C (in ppm) by

$$\log_{10} C = m \cdot \log_{10}(R_s / R_0) + b \qquad (3)$$

where m and b are constant coefficients obtained from the sensitivity graph for the specific gas.

VI. SOFTWARE AND ALERT LOGIC

A. Control Flow

The software works according to the diagram in Fig. 9. After startup, the sensors are initialized and the threshold limits set. The device then enters an infinite loop of reading sensor values, formatting them and testing them against the threshold limits. When any threshold is exceeded, the alarm (buzzer and red LED) turns on prior to data transmission, while if none is exceeded, the data is transmitted. In either case the data is transmitted using an HTTP request to ThingSpeak, ensuring that local alarms never delay or prevent cloud updating.

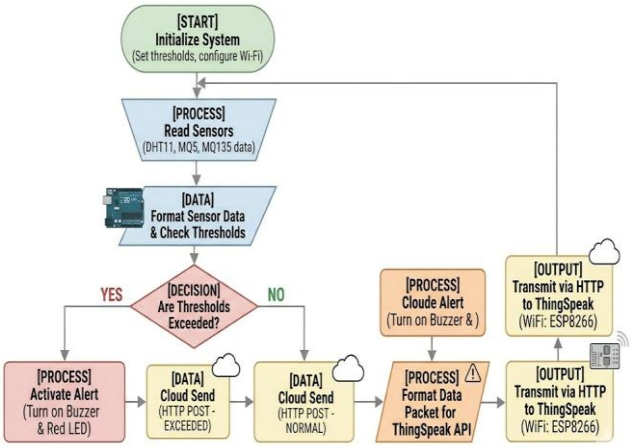


Fig. 9. Algorithm flowchart of the monitoring node.

B. Data Transmission

Readings are uploaded with HTTP POST requests to the ThingSpeak API. The free ThingSpeak tier accepts one update per 15 s per channel, which was therefore used as the sampling period. Four data fields were configured (Table II).

TABLE II
THINGSPEAK CHANNEL FIELDS

Field	Quantity	Unit
Field 1	MQ5 gas reading (analog)	ADC counts
Field 2	Temperature (DHT11)	°C
Field 3	Relative humidity (DHT11)	%
Field 4	MQ135 air-quality / ammonia reading	ppm (est.)

C. AQI Estimation

In order to translate the data into terms that could be readily understood by a layman, the readings were converted into the AQI. For a pollutant p with measured concentration C_p, the sub-index is obtained by linear interpolation between the breakpoints of Fig. 2,

$$I_p = [(I_{HI} - I_{LO})/(BP_{HI} - BP_{LO})]\,(C_p - BP_{LO}) + I_{LO} \qquad (4)$$

where BP_LO and BP_HI are the breakpoints enclosing C_p and I_LO and I_HI are the corresponding index values. The overall index is the largest of the sub-indices,

$$AQI = \max(I_1, I_2, \ldots, I_n) \qquad (5)$$

Since the node does not have a calibrated particulate sensor channel, the reported AQI is estimated based solely on gas channels, and should be considered informative rather than regulatory.

VII. RESULTS AND DISCUSSION

A. System Operation

The prototype was tested under live conditions. After initialization, the node initializes its sensors, joins the local network via mobile hotspot and starts uploading data to ThingSpeak. Manual intervention was not required after that.

B. Cloud Visualization

Fig. 10 shows the live temperature widget on the ThingSpeak dashboard, and the logged traces are shown in Figs. 11 and 12 for a roughly 15-minute window.



Fig. 10. Current room temperature shown on the ThingSpeak numeric display.

MQ5 channel (Field 1) in Fig. 11 is relatively constant until it reaches about 600 after being around 550 and drops suddenly in the end, while temperature channel (Field 2) alternates between 24 and 25 degrees Celsius – which is just one degree resolution of DHT11 crossing by the small changes in actual temperature. In Fig. 12 the humidity (Field 3) is almost constantly at 60% with a small increase to 65%, and the MQ135 channel (Field 4) fluctuates between 430 and 480 with a maximum at the end of the plot. The brief excursions on several channels at the same moment are consistent with a real disturbance near the node rather than independent sensor noise.

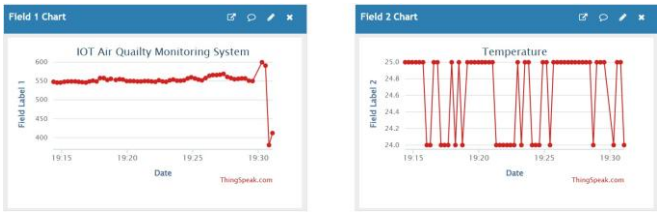


Fig. 11. ThingSpeak logs: Field 1 (MQ5 gas reading) and Field 2 (temperature in °C) over time.

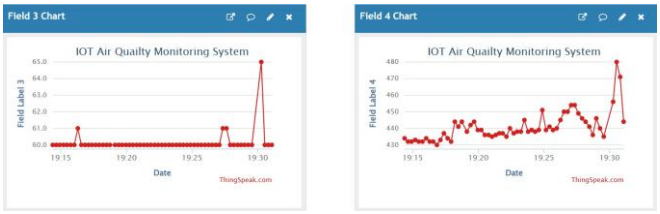


Fig. 12. ThingSpeak logs: Field 3 (relative humidity, %) and Field 4 (MQ135 air-quality / ammonia reading) over time.

C. Alert Validation

To verify the alert functionality, sensors were subjected to an increased concentration of gas. Once the value passed the threshold, the buzzer rang and the LED turned on, and the values continued uploading to the cloud while the alarm was sounding. Alerts were therefore available both locally and remotely.

D. Accuracy Against Reference Values

Table III compares values measured by the node with expected reference values. The temperature discrepancy was 0.2 °C, and that of AQI was 2 index units; the humidity level was 2 percentage points more than expected, which falls within the ±5 % relative humidity range mentioned for the DHT11 sensor. The raw values of the MQ135 and MQ5 sensors were 312 and 148 counts, respectively. Values marked as estimated were derived from the datasheet curves and are therefore approximate.

TABLE III
MEASURED VERSUS EXPECTED VALUES

Parameter	Expected	Measured	Unit	Remark
AQI	60	62	index	Very close
Temperature	35	34.8	°C	Accurate
Humidity	24	26	%	Slight variation
CO_2 (MQ135)	420–480	~450	ppm	Within range
CO	0.5–1.5	~1.0 (est.)	ppm	Safe level
NH_3	5–20	~10 (est.)	ppb	Very low
LPG / CH_4	1–5	~3 (est.)	ppm	No leakage
MQ135	n/a	312	ADC	Air quality
MQ5	n/a	148	ADC	Gas sensor

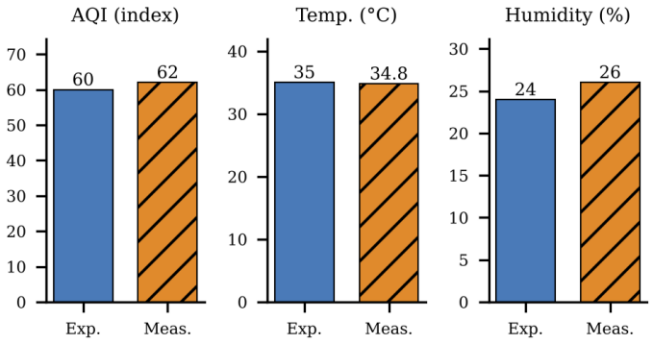


Fig. 13. Expected and measured values of AQI, temperature and humidity (data from Table III).

The AQI was also tracked over several days against the value published by a standard source (Fig. 14). The two series rise and fall together, and the node's value is usually within a few points of the published one. The gap is not surprising: the node uses inexpensive MQ sensors whose outputs are approximate and affected by temperature, humidity and the age of the heating element, whereas a monitoring station uses calibrated instruments.

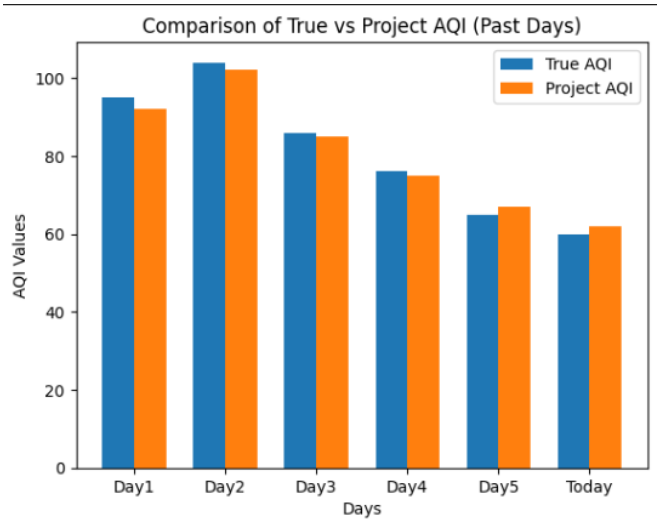


Fig. 14. Comparison of the published (true) AQI and the AQI calculated by the proposed system over the past few days.

E. Limitations

Four limitations should be stated plainly. First, MQ sensors are not selective, so MQ135 and MQ5 readings reflect a mixture of gases and the individual concentrations are estimates. Second, the DHT11 resolves only whole degrees and has a modest accuracy. Third, calibration used datasheet constants and was not checked in a gas chamber. Fourth, because the dust sensor in the prototype was not calibrated using a reference device, particulate measurements have not been considered for the AQI calculation. In addition, the test run was short and no investigation of long-term drift was conducted.

VIII. SMART CITY APPLICATION PERSPECTIVE

While a single sensor node gives information about one point; a city requires multiple. Fig. 15 sketches how copies of the node could be placed at traffic junctions, residential blocks, industrial areas and sensitive sites such as schools and hospitals, all reporting to one cloud channel group. This section is a design proposal and was not tested at that scale.

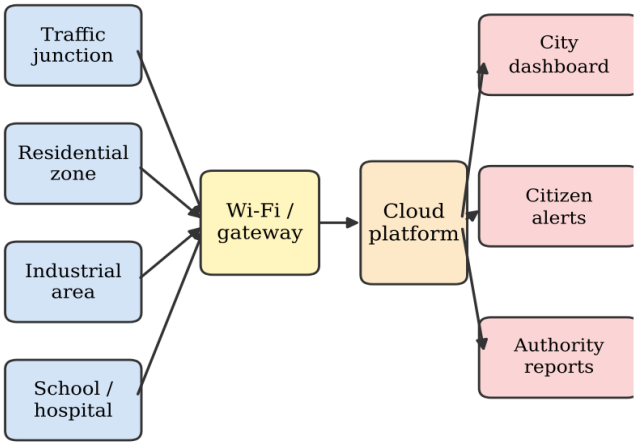


Fig. 15. Deployment of multiple sensor nodes for the air quality layer of the smart city.

- Scalability: The individual node will not be reliant on any other infrastructure but would require only Wi-Fi and power supply. To expand its reach, the node can even be scaled up to support LoRaWAN communication without impacting the sensing functionality.
- Utility of data: The measuring of several districts can serve as an input for the city dashboard and create alerts for the public, as well as information for decision-making, such as routing or construction.
- Citizen participation: because the hardware is cheap, schools and residents' associations can run their own nodes, which widens coverage beyond what an authority could install.
- Accuracy of data: low-cost nodes should be periodically synchronized with a reference station and re-calibrated

IX. CONCLUSION AND FUTURE WORK

In this project, an IoT air pollution sensor node was designed and developed with the help of Arduino Uno, DHT11, MQ5 and MQ135 sensors and ESP8266 connectivity module to ThingSpeak cloud. The developed prototype was able to give continuous measurements on the dashboard and also local alarm if any threshold exceeded and was in agreement with the reference values of temperature and AQI to a good extent, which made it suitable for awareness purposes.

Future work would include adding a particulate matter channel with calibration, testing the sensors against a reference station, machine learning algorithm for pollution episode prediction and a mobile application for citizens.

REFERENCES

[1] World Health Organization, "Ambient (outdoor) air pollution," WHO Fact Sheet, Geneva, Switzerland.

[2] World Health Organization, "Air pollution and child health: Prescribing clean air," Geneva, Switzerland, 2018.

[3] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," Future Gener. Comput. Syst., vol. 29, no. 7, pp. 1645–1660, Sep. 2013.

[4] A. R. Kondeti, L. Rudraksha, S. Chinnaiahgari, and A. Bujunuru, "Real-time air quality and weather monitoring system utilizing IoT for sustainable urban development and environmental management," Eng. Proc., vol. 118, no. 1, p. 56, Nov. 2025.

[5] A. Zanella, N. Bui, A. Castellani, L. Vangelista, and M. Zorzi, "Internet of Things for smart cities," IEEE Internet Things J., vol. 1, no. 1, pp. 22–32, Feb. 2014.

[6] J. J. Caubel et al., "A new black carbon sensor for dense air quality monitoring networks," Sensors, vol. 18, no. 3, 2018.

[7] G. Lo Re et al., "Mobile application for air pollution exposure," IEEE Sensors J., vol. 13, no. 10, pp. 3965–3974, 2013.

[8] A. Jayakumar, "IoT based air pollution monitoring system," Int. Res. J. Eng. Technol., 2021.

[9] H. Gupta et al., "IoT based air pollution monitoring for smart cities," in Proc. Int. Conf. Comput. Commun. Autom., 2019, pp. 1–5.

[10] M. Singh et al., "IoT based air monitoring using Arduino," Int. Res. J. Eng. Technol., vol. 6, no. 3, pp. 1–4, 2019.

[11] N. D. Desai et al., "IoT monitoring using BeagleBone Black," in Proc. Int. Conf. Trends Electron. Informat., 2017, pp. 1–6.

[12] K. S. E. Phala et al., "Air quality monitoring system standards," in Proc. IEEE Int. Conf. Ind. Technol., 2016, pp. 2038–2043.

[13] P. Pal et al., “IoT pollution monitoring system,” Int. Res. J. Eng. Technol., vol. 4, no. 5, pp. 1–5, 2017.

[14] S. Dhingra et al., “IoT mobile air monitoring system,” IEEE Internet Things J., vol. 6, no. 3, pp. 5577–5584, 2019.

[15] M. Anitha and L. S. Kumar, “Development of an IoT-enabled air pollution monitoring and air purifier system,” MAPAN, vol. 38, pp. 1–12, May 2023.

[16] Central Pollution Control Board, “National Air Quality Index,” Ministry of Environment, Forest and Climate Change, Government of India, New Delhi, India, 2014.

[17] Hanwei Electronics, “MQ-135 gas sensor technical data sheet,” Zhengzhou, China.

[18] Hanwei Electronics, “MQ-5 gas sensor technical data sheet,” Zhengzhou, China.

[19] MathWorks, “ThingSpeak IoT analytics platform documentation,” Natick, MA, USA.